

FULL STACK DEVELOPER SYLLABUS

Real-World Ready · 18 Weeks · Build & Ship Actual Projects

PRE-PHASE · Non-Negotiable Foundations

Days 1–5

JavaScript Core

- let/const/scope, functions (arrow + normal)
- Objects, Arrays, destructuring
- map, filter, reduce
- Promises & async/await
- try/catch error handling
- Basic time complexity

How Internet Works

- HTTP, REST, JSON basics
- Browser → Server → DB → Response
- Status codes (200/400/401/500)
- Event Loop & call stack
- HTTPS vs HTTP

Auth & DB Concepts

- SQL vs NoSQL
- MongoDB collections & documents
- Authentication vs Authorization
- Password hashing (why?)
- JWT tokens concept
- TypeScript: types & interfaces basics

✓ **CHECKPOINT:** Explain async/await, REST API, JWT, SQL vs NoSQL without looking. Confident? → Phase 1.

PHASE 1 · Backend Engineering

Weeks 1–3

Node.js + Express

- Event loop in real code
- Async/await & error handling patterns
- Environment variables (.env) + Zod validation
- MVC folder structure
- Routes → Controllers → Services layer
- Middleware (custom + built-in)
- CORS setup + Rate limiting

Authentication System

- User schema design
- bcrypt password hashing
- JWT generation & refresh tokens
- Token verification middleware
- Role-based access control (RBAC)
- Logout & token blacklist strategy
- helmet.js + XSS + input sanitization

File Handling & Webhooks

- File uploads (Multer + Cloudinary/S3)
- Webhook handling + signature verification
- Centralized error handling middleware

✓ **CHECKPOINT:** Build full auth API — register, login, protected route, RBAC, file upload, security headers. Test with Postman. No tutorial.

PHASE 2 · Database Engineering

Weeks 2–5 (Overlap)

MongoDB Mastery

- Schema design: normalize vs embed
- Mongoose models & validation
- Indexing (single, compound, text)
- Query optimization techniques
- Aggregation (\$match, \$group, \$lookup)
- Pagination & filtering patterns
- MongoDB Atlas + connection pooling

Redis (Real Projects Use This)

- What is Redis & when to use it
- Cache-aside pattern (API caching)
- Session/token storage in Redis
- Cache invalidation strategy
- Rate limiting with Redis

SQL Awareness

- SELECT, JOIN, WHERE, GROUP BY
- Indexes in SQL
- When to use SQL vs NoSQL

✓ **CHECKPOINT:** Design flagship project schema. Write aggregation queries. Add Redis caching to one route. Explain indexing choices aloud.

PHASE 3 · Frontend with React

Weeks 4–8

React Core • JSX & component thinking • Props, state, lifting state up • useState, useEffect, useRef • useMemo, useCallback, custom hooks • Context API + Zustand (global state) • React Query / TanStack Query • Form handling (React Hook Form)	React Advanced + Integration • Protected routes & auth guard • Role-based dashboard rendering • API integration (Axios / Fetch) • File upload UI (progress + preview) • Real-time UI (WebSocket/Socket.io) • TypeScript in React (props typing) • Error boundaries & loading states • Feature-based folder structure
✓ CHECKPOINT: Full dashboard — login, protected routes, role views, file upload, real-time update, API-connected. UI clean & functional. No tutorial.	

PHASE 4 · AI Integration Weeks 8–13	
LLM API (Backend) • OpenAI / Gemini API setup (server-side only) • API key management • Structured JSON response prompting • System prompt architecture • Context window & token cost awareness • Streaming responses (SSE / WebSocket) • Error fallback & retry logic • AI response caching (Redis)	AI Features to Build • AI quiz / content generator • AI feedback engine • Semantic search (embeddings + vector DB) • RAG — Retrieval Augmented Generation basics • LangChain basics (awareness level)
✓ CHECKPOINT: Ship one AI feature — backend LLM call → frontend display. Stream response, cache with Redis, handle errors. Explain full AI flow.	

PHASE 5 · Production & Deployment Weeks 12–16 (Overlap)	
DevOps Essentials • Git workflow (branching, commits, PRs) • Frontend deploy (Vercel / Netlify) • Backend deploy (Render / Railway) • MongoDB Atlas production setup • Docker basics (Dockerfile + compose) • Containerize Node + Redis app • CI/CD basics (GitHub Actions) • HTTPS, domain, SSL certificate	Testing (Basics Only) • Production error logging (Winston/Sentry) • Health check endpoint • Unit tests with Jest (basics) • API integration tests (Supertest) • README & API documentation (Swagger)
✓ CHECKPOINT: Project fully deployed. Frontend + Backend + DB + AI + Redis all live. Docker builds. Share the URL.	

PHASE 6 · Polish & Real-World Depth Weeks 16–18	
Code Quality • Refactor to clean architecture • API documentation (Swagger/Postman) • Security audit (helmet, sanitize, HTTPS) • Socket.io real-time feature (complete) • Stripe/payment webhook end-to-end	Explain Like a Developer • Whiteboard auth flow (JWT lifecycle) • Explain DB schema design decisions • Explain AI integration & prompt strategy • Explain Redis caching strategy • Walk through codebase confidently • Handle 'why did you do it this way?'

✓ **FINAL CHECKPOINT:** Record yourself explaining the entire system for 5 minutes — auth, DB, AI, Redis, Docker, deployment. No notes. → Real-World Ready.

SKIP COMPLETELY (Not Needed for Real Projects at This Stage)

✗ Kubernetes	✗ Microservices Architecture	✗ Deep ML / Math
✗ Advanced TypeScript Generics	✗ GraphQL (learn after basics solid)	✗ Fancy animations / 3D

REALISTIC TIMELINE

Phase	Focus	Duration
Pre-Phase	JS, HTTP, Auth & DB concepts	5 days
Phase 1	Express + Auth API + Security	2–3 weeks
Phase 2	MongoDB + Redis + SQL Basics	2–3 weeks
Phase 3	React + Full Frontend Integration	3–4 weeks
Phase 4	AI Feature (LLM + RAG + Streaming)	4–5 weeks
Phase 5	Docker + Deploy + Tests + CI/CD	3–4 weeks
Phase 6	Polish + Real-World Depth	2 weeks
TOTAL	One solid full-stack + AI project, shipped & explainable	~18 Weeks

Weekly Habit: Backend progress · Frontend progress · AI progress · Refactoring · Documentation